

บทที่ 13 : เธรด (Thread)

บรรยายโดย ผศ.ดร.ธราวิเชษฐ์ ธิติจรูญโรจน์

คณะเทคโนโลยีสารสนเทศ

สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

ตัวอย่าง

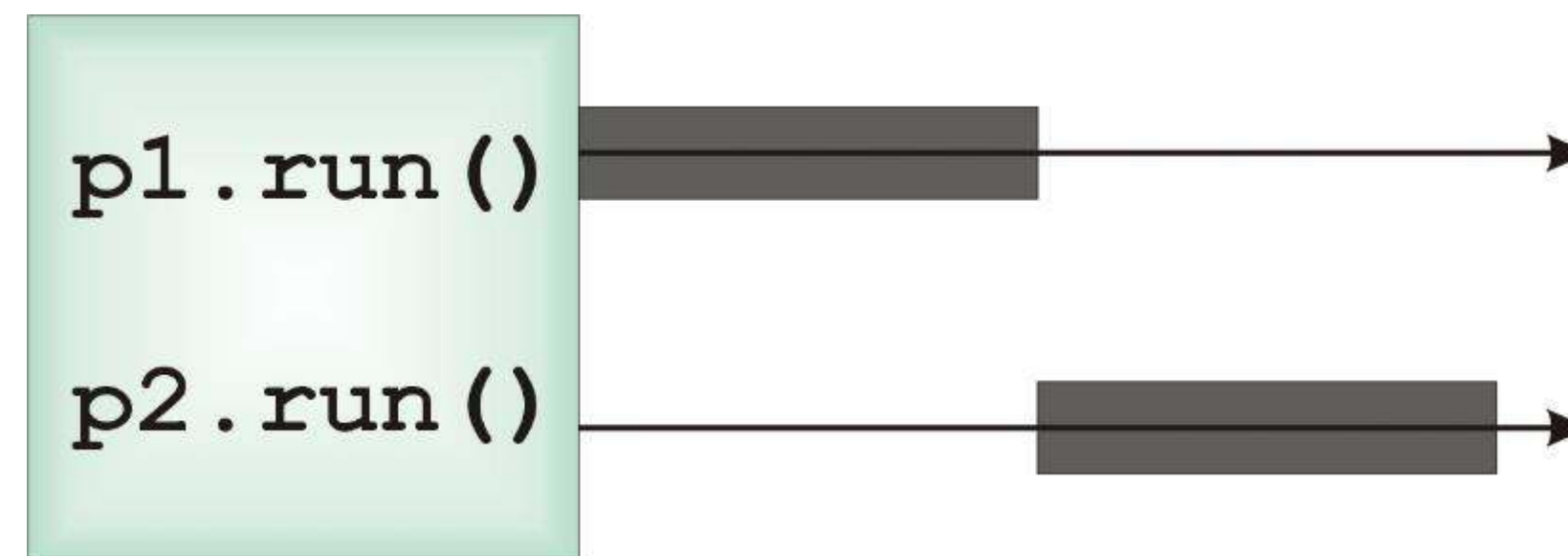
```
class PrintName {  
    String name;  
    public PrintName(String n) {    name = n;    }  
    public void run() {  
        for(int i=0; i<100; i++)  
            System.out.println(name) ;  
    }  
}  
  
public class TestPrintName {  
    public static void main(String args[]) {  
        PrintName p1 = new PrintName("Thana") ;  
        PrintName p2 = new PrintName("Somchai") ;  
        p1.run() ;  
        p2.run() ;  
    }  
}
```

Output:

Thana
Thana
Thana
Thana
Thana
Somchai
Somchai
Somchai
Somchai
Somchai

อธิบายตัวอย่างโปรแกรม

- โปรแกรมนี้เป็นการทำงานใน*รูปแบบปกติ* โดยคลาส PrintName มีเมธอด run() ซึ่งจะพิมพ์ข้อความในคุณลักษณะ name จำนวน 5 ครั้ง
- คลาส TestPrintName จะสร้างออบเจ็คของคลาส PrintName ขึ้นมา 2 ออบเจ็คที่ชื่อ p1 และ p2 จากนั้นจะเรียกใช้เมธอด run() เพื่อพิมพ์ข้อความออกมา
- เนื่องจากโปรแกรมนี*ไม่ได้เป็นโปรแกรมแบบเธรด* ดังนั้น ซีพียูจะทำคำสั่ง p1.run() ให้เสร็จก่อน แล้วจึงจะทำคำสั่ง p2.run()



ระบบปฏิบัติการแบบ Multitasking

- ระบบปฏิบัติการแบบ Multitasking คือระบบปฏิบัติการที่อนุญาตให้ผู้ใช้สามารถส่งโปรแกรมให้ **ระบบปฏิบัติการทำงานได้มากกว่าหนึ่งโปรแกรมพร้อมกัน** ซึ่งโปรแกรมแต่ละโปรแกรมที่รันอยู่ในระบบปฏิบัติการจะสร้าง process ขึ้นมา
- ระบบปฏิบัติการแบบ Multitasking จะมี process หลายๆ process เข้าคิวรอการทำงาน โดยระบบปฏิบัติการจะกำหนดลำดับของการทำงานของ process เอง

เธรดคืออะไร

① task

② Application / system

③ Process

Code Data Files

Thread

Registers

stack



Process

Code Data Files

Thread

Registers

stack



Thread

Registers

stack



เธรด

คือ ส่วนเดียวและเล็กที่สุดของ ^{โปรแกรมย่อย 1 job} Process ที่สามารถทำงานพร้อมกันกับส่วนอื่น ๆ (เธรดอื่น) ของ Process เดียวกันได้ ซึ่งแต่ละ Process อาจจะมีเธรดเดียวหรือหลายเธรดก็ได้

มัลติเธรด

คือ กระบวนการของการดำเนินการหลายเธรดพร้อมกัน

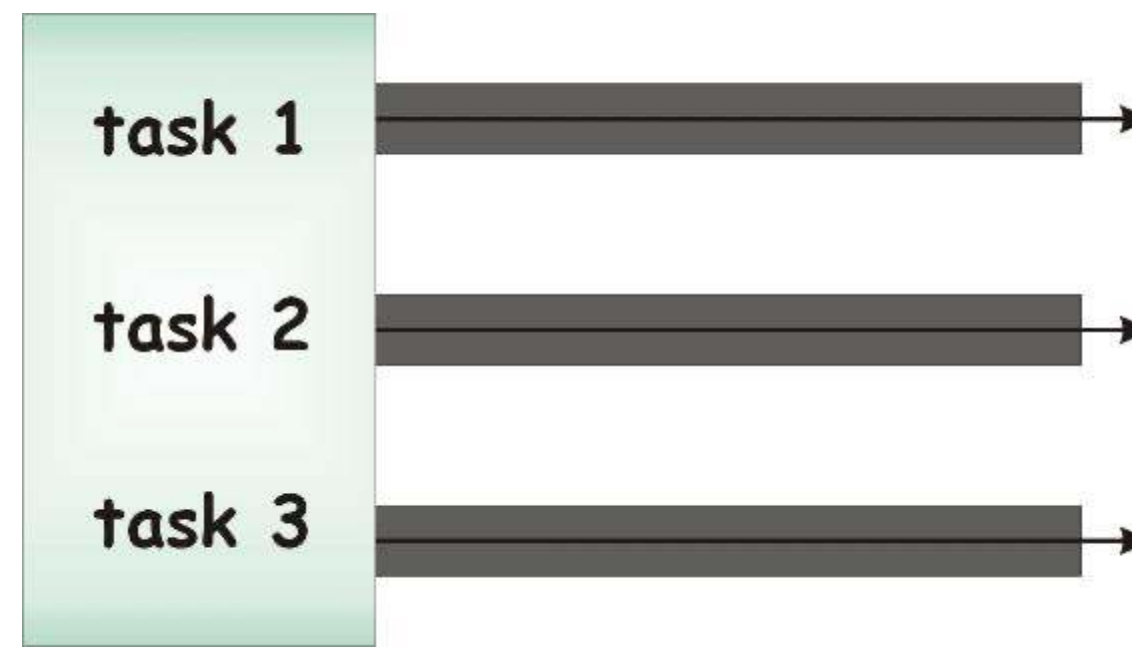
โปรแกรมมัลติเธรด

72

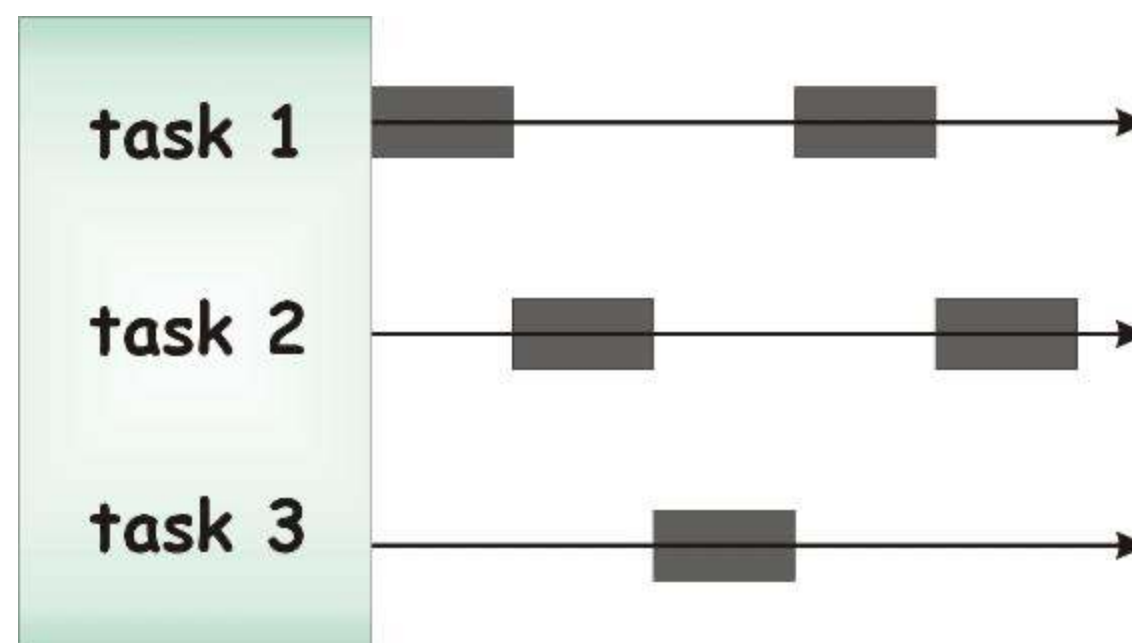
- โปรแกรมมัลติเธรดจะเป็นการทำงานหลายงานพร้อมกันโดยแต่ละงานจะเรียกว่าเธรด ซึ่งจะแตกต่างจาก process ที่ทำงานภายใต้ระบบปฏิบัติการแบบ Multitasking ตรงที่ process แต่ละ process จะมีความเป็นอิสระต่อกัน แต่เธรดแต่ละเธรดอาจจะใช้ข้อมูลร่วมกันซึ่งเปรียบเสมือนชีฟิยูจำลอง (Virtual CPU)
- โปรแกรมแบบเธรดจะมีหลักการทำงานที่แตกต่างกัน ทั้งนี้ขึ้นอยู่กับระบบปฏิบัติการและจำนวนชีฟิยู

โปรแกรมมัลติเธรด

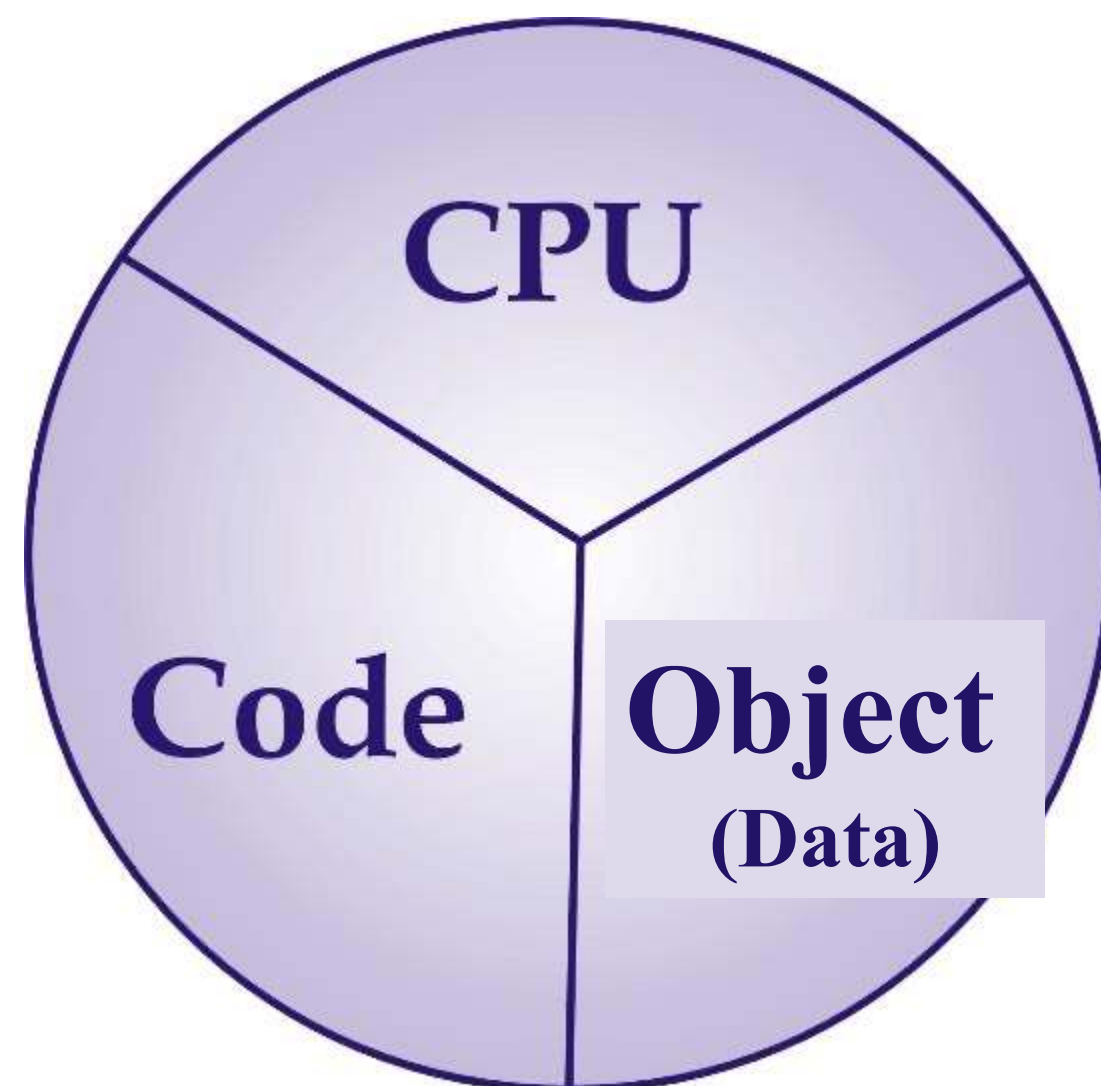
ในระบบคอมพิวเตอร์ที่มีซีพียูหลายตัว โปรแกรมแบบเธรดแต่ละโปรแกรมสามารถที่จะรันพร้อมกันได้



ในระบบคอมพิวเตอร์ที่มีซีพียูตัวเดียว ระบบปฏิบัติการจะเป็นส่วนที่จะจัดการตารางการทำงานของโปรแกรมเธรด ซึ่งอาจแบ่งเวลาการทำงานของซีพียู



องค์ประกอบของโปรแกรมแบบเธรด



- ซีพียู คือ *การจัดการทำงานของโปรแกรมแบบเธรด*
- ซอร์สโค้ด คือ คลาสในภาษาจาวาที่เป็น*คลาสแบบเธรด*
- ออปเจ็ค คือ *ออปเจ็ค*ของคลาสแบบเธรด

ออปเจ็คแบบเธรด



- โปรแกรมภาษาจาวาสามารถกำหนดให้ออปเจ็คของคลาสใด ๆ ทำงานเป็นแบบเธรดได้ ซึ่งทำให้ *สามารถรันโปรแกรมของออปเจ็คแบบเธรดหลาย ๆ ออปเจ็คพร้อมกันได้*
- ภาษาจาวาจะมีตัวจัดตารางเวลา (Thread Scheduler) ของออปเจ็คแบบเธรด เพื่อ *จัดลำดับการทำงาน* ของออปเจ็คแบบเธรด
- โปรแกรมบางประเภทจำเป็นต้องพัฒนาเป็นแบบเธรด อาทิเช่น
 - โปรแกรมภาพกราฟฟิกแบบเคลื่อนไหว (Graphics Animation)
 - โปรแกรมนาฬิกา
 - โปรแกรมแม่ข่าย (Application Server)

การสร้างคลาสแบบเธรด

โปรแกรมแบบเธรดในภาษาจาวาจะใช้หลักการของ*การสร้างออบเจ็คของคลาสแบบเธรด*แล้วเรียกใช้เมธอดเพื่อส่งให้ออบเจ็คดังกล่าวทำงานพร้อมกัน ซึ่งคลาสแบบเธรดในภาษาจาวา คือ

- คลาสที่สืบทอดมาจากคลาสที่ชื่อ Thread

```
public class Poring extends Thread {  
    ...  
}
```

- คลาสที่ implements อินเตอร์เฟส Runnable

```
public class Poring implements Runnable {  
    ...  
}
```

ออบเจ็คที่สร้างมาจากคลาสแบบเธรดจะเรียกว่า*ออบเจ็คแบบ runnable*

การสร้างคลาสแบบเธรด

โปรแกรมแบบเธรดในภาษาจาวาจะใช้หลักการของ*การสร้างออบเจ็คของคลาสแบบเธรด*แล้วเรียกใช้เมธอดเพื่อส่งให้ออบเจ็คดังกล่าวทำงานพร้อมกัน ซึ่งคลาสแบบเธรดในภาษาจาวา คือ

- คลาสที่สืบทอดมาจากคลาสที่ชื่อ Thread

```
public class Poring extends Thread {  
    ...  
}
```

- คลาสที่ implements อินเตอร์เฟส Runnable

```
public class Poring implements Runnable {  
    ...  
}
```

ออบเจ็คที่สร้างมาจากคลาสแบบเธรดจะเรียกว่า*ออบเจ็คแบบ runnable*

การ implements อินเตอร์เฟส Runnable

คลาสแบบธรรมดาสามารถสร้างได้โดยการ implements อินเตอร์เฟส Runnable ซึ่งเมธอดเดียวที่จะต้องเขียนบล็อกคำสั่ง คือ *เมธอด run()* โดยมีรูปแบบดังนี้

```
public class ClassName implements Runnable {  
    public void run() {  
        [statements]  
    }  
}
```

คำสั่งที่อยู่ในเมธอด run() คือ *ชุดคำสั่งที่ต้องการให้โปรแกรมทำงานในลักษณะแบบธรรมดา*

ตัวอย่างคลาสที่ implements อินเทอร์เฟซ Runnable



```
public class PrintName implements Runnable {  
    public String name;  
    public PrintName(String n) { name = n; }  
    public void run() {  
        for(int i=0; i< 5; i++)  
            System.out.println(name + " : " + i);  
    }  
}
```

การ implements อินเตอร์เฟส Runnable

โปรแกรมจาวาที่รันแอปเจ็คของคลาสแบบธรรมดาในลักษณะนี้จะต้องมีการสร้างแอปเจ็คของคลาสที่ชื่อ Thread ก่อน ซึ่ง Constructor ของคลาสที่ชื่อ Thread จะมี argument เพื่อรับแอปเจ็คของคลาสที่ implements อินเตอร์เฟส Runnable โดยมีรูปแบบดังนี้

ถ้า obj เป็น Thread

```
public Thread(Runnable obj)
```


การ implements อินเตอร์เฟส Runnable

หลังจากที่สร้าง **ออปเจ็คของคลาสแบบเธรด** แล้ว จะสามารถสั่งให้ออปเจ็ค obj เริ่มทำงานได้โดยการเรียกใช้เมธอด start() ที่อยู่ในคลาสที่ชื่อ Thread ซึ่ง **คลาส Thread จะลงทะเบียนออปเจ็คดังกล่าวลงในตัวจัดตารางเวลา** และเมื่อ **ซีพียูเรียกรันโปรแกรมของออปเจ็คดังกล่าวตามคำสั่งในเมธอด run()**

```
public class PrintNameThread {  
    public static void main(String args[]) {  
        PrintName p1 = new PrintName("Thana");  
        Thread t1 = new Thread(p1);  
        t1.start();  
    }  
}
```

↓ Runnable
↓ ลงทะเบียนใน Thread
↓ เรียกใช้

กล่าวคือ เมื่อ Thread ถูกลงทะเบียนผ่านคำสั่ง start() แล้ว CPU จะเรียกใช้งานคำสั่ง run() ใน **ออปเจ็คของคลาสแบบเธรดเอง**

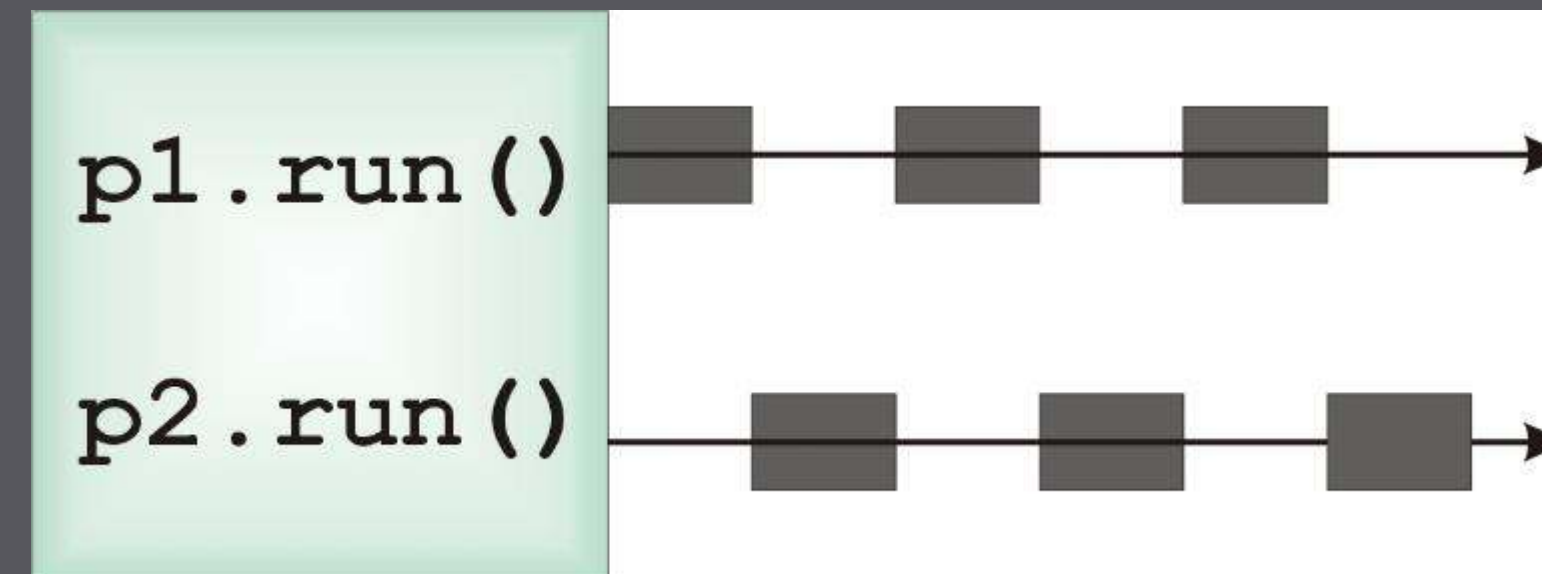
ตัวอย่าง

```

public class PrintName implements Runnable {
    public String name;
    public PrintName(String n) { name = n; }
    public void run() {
        for(int i=0; i<100; i++)
            System.out.println(name + " : " + i);
    }
}

public class PrintNameThread {
    public static void main(String args[]) {
        PrintName p1 = new PrintName("Thana");
        PrintName p2 = new PrintName("Somchai");
        Thread t1 = new Thread(p1);
        Thread t2 = new Thread(p2);
        t1.start();
        t2.start();
    }
}

```



Output - Lab12 (run) ×

```

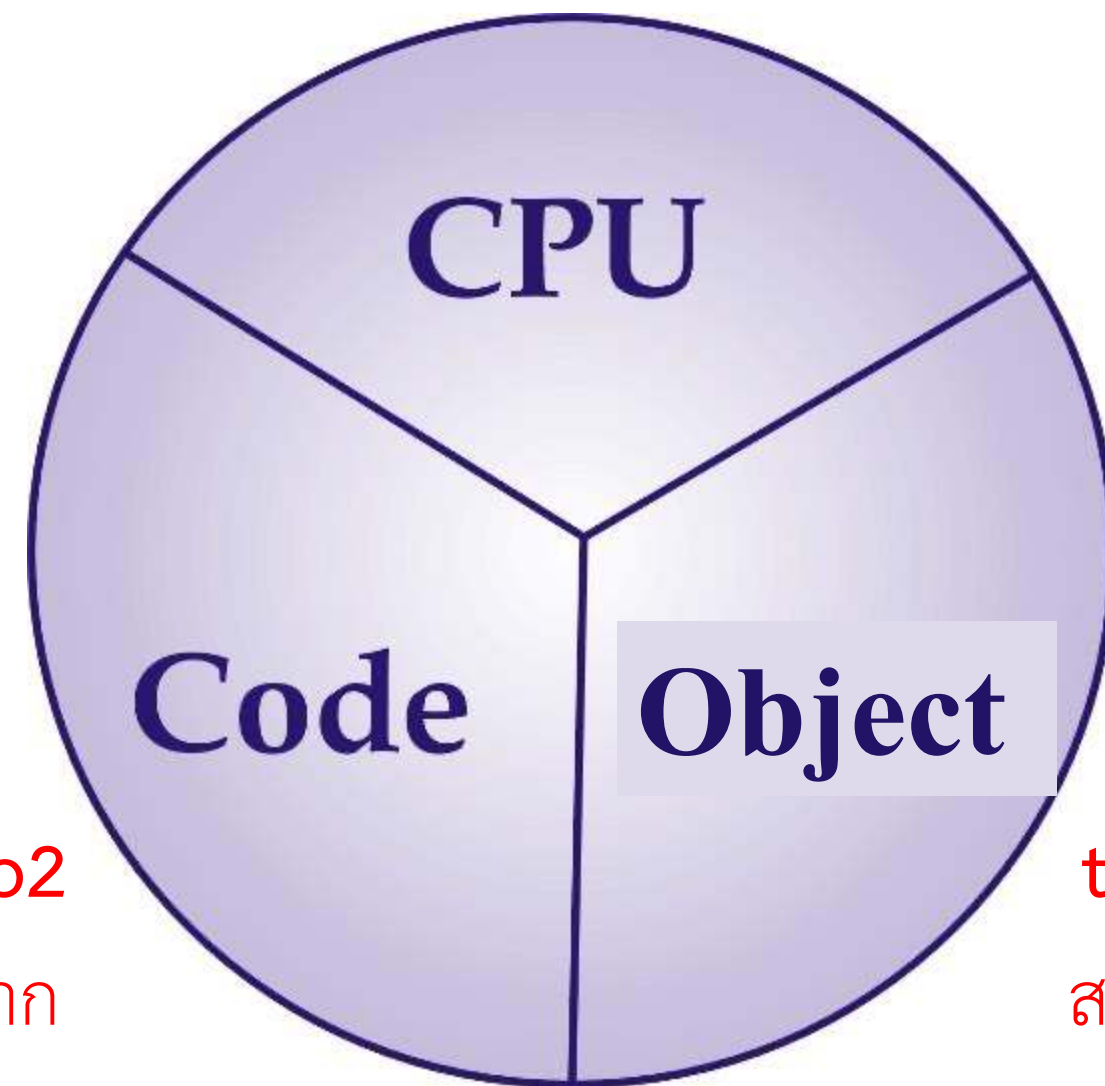
run:
Thana : 0
Thana : 1
Thana : 2
Thana : 3
Thana : 4
Thana : 5
Thana : 6
Thana : 7
Thana : 8
Thana : 9
Thana : 10
Thana : 11
Thana : 12
Thana : 13
Somchai : 0
Thana : 14
Thana : 15
Thana : 16
Thana : 17
Thana : 18
Thana : 19
Somchai : 1
Somchai : 2
Somchai : 3
Somchai : 4
Somchai : 5

```

Handwritten annotations in blue ink show that the first 13 lines of output (Thana : 0 to Thana : 13) are grouped together with a bracket and a '+1' annotation. The next 7 lines (Thana : 14 to Thana : 19) are grouped together with a bracket and a '+2' annotation. The final 5 lines (Somchai : 0 to Somchai : 5) are grouped together with a bracket and a '+2' annotation.

อธิบายตัวอย่างโปรแกรม

ตัวจัดตารางเวลา
(Thread Scheduler)
โดยอาศัยคำสั่ง `t1.start()` และ `t2.start()`



p1 และ p2
สร้างมาจาก
class
`PrintName`
`implements`
`Runnable`

t1 และ t2
สร้างมาจาก
คลาส `Thread`

- Code

ตัวอย่างโปรแกรมนี้เป็นการปรับปรุงคลาส `PrintName` ให้เป็น
คลาสแบบเธรด โดยการ *implements อินเทอร์เฟซ `Runnable`*

- Object

ซึ่งโปรแกรม `PrintNameThread` สร้างออบเจกต์ของคลาส
`PrintName` ขึ้นมา 2 ออบเจกต์ คือ p1 และ p2 ซึ่งเป็นออบเจกต์
แบบ `Runnable` จากนั้น *โปรแกรมได้สร้างออบเจกต์ของคลาส*
Thread คือ t1 และ t2 ที่มีซอร์สโค้ดเดียวกัน คือ `PrintName` แต่
มีออบเจกต์ต่างกันคือ p1 และ p2

- CPU

เมธอด `t1.start()` และ `t2.start()` เป็นการส่งออบเจกต์แบบเธรดเข้าไป
จองคิวกับตัวจัดตารางเวลา

การสร้างคลาสแบบเธรด

โปรแกรมแบบเธรดในภาษาจาวาจะใช้หลักการของ*การสร้างออปเจ็คของคลาสแบบเธรด*แล้วเรียกใช้เมธอดเพื่อส่งให้ออปเจ็คดังกล่าวทำงานพร้อมกัน ซึ่งคลาสแบบเธรดในภาษาจาวา คือ

- คลาสที่สืบทอดมาจากคลาสที่ชื่อ Thread

```
public class Poring extends Thread {  
    ...  
}
```

- คลาสที่ implements อินเตอร์เฟส Runnable

```
public class Poring implements Runnable {  
    ...  
}
```

ออปเจ็คที่สร้างมาจากคลาสแบบเธรดจะเรียกว่า*ออปเจ็คแบบ runnable*

การสืบทอดคลาสที่ชื่อ Thread

คลาสแบบเธรดสามารถสร้างได้โดยการสืบทอดคลาสที่ชื่อ Thread ซึ่ง *จะต้องมีการ override เมธอด run()* โดยมีรูปแบบดังนี้

** เป็น Thread*

```
public class ClassName extends Thread {  
    public void run(){ // overridden  
        [statements]  
    }  
}
```

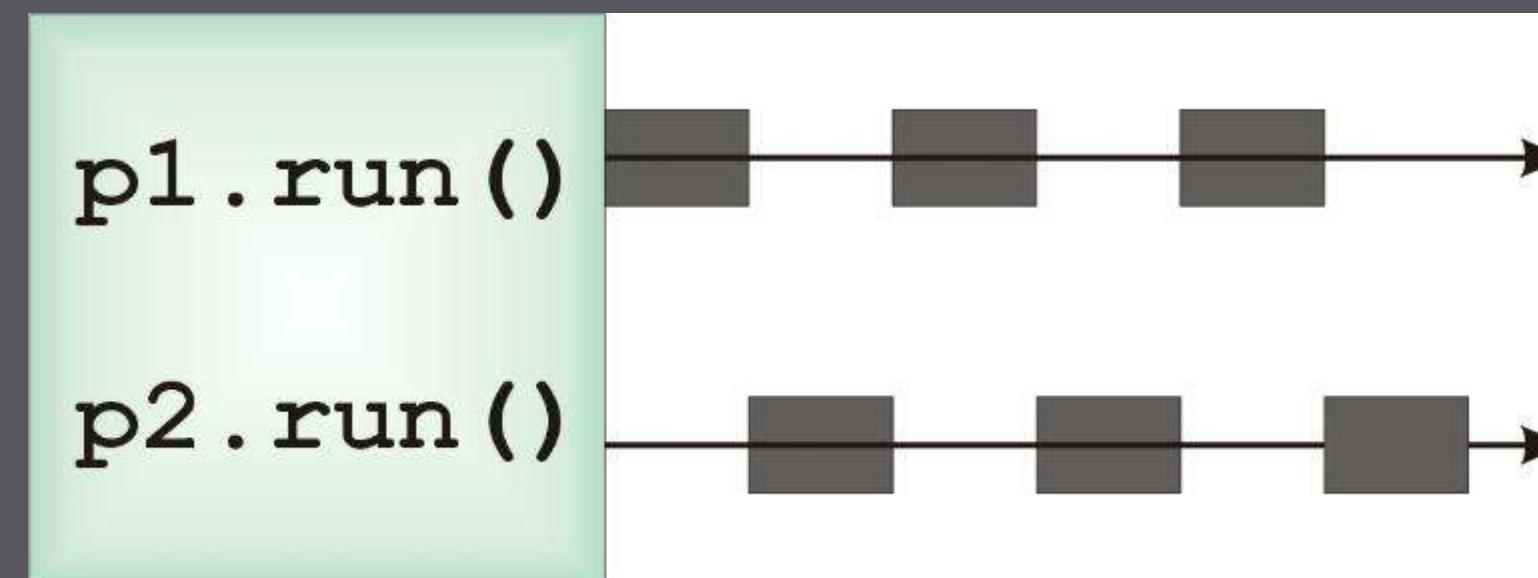
นักศึกษาสามารถเรียกใช้เมธอด *start()* ได้โดยตรง เพื่อลงทะเบียนออปเจ็คของคลาสดังกล่าวในตัวจัดตารางเวลาได้ โดยไม่จำเป็นต้องสร้างออปเจ็คของคลาส Thread และเมื่อซีพียูเรียกรันโปรแกรมของออปเจ็ค คำสั่งในเมธอดแบบ overridden ที่ชื่อ run() จะถูกส่งงานต่อไป

ตัวอย่าง

```

class PrintName extends Thread {
    public String name;
    public PrintName(String n) { name = n; }
    public void run() {
        for(int i=0; i<100; i++)
            System.out.println(name + " : " + i);
    }
}

public class PrintNameThread {
    public static void main(String args[]) {
        PrintName p1 = new PrintName("Thana");
        PrintName p2 = new PrintName("Somchai");
        p1.start();
        p2.start();
    }
}
  
```



Output - Lab12 (run) ×

```

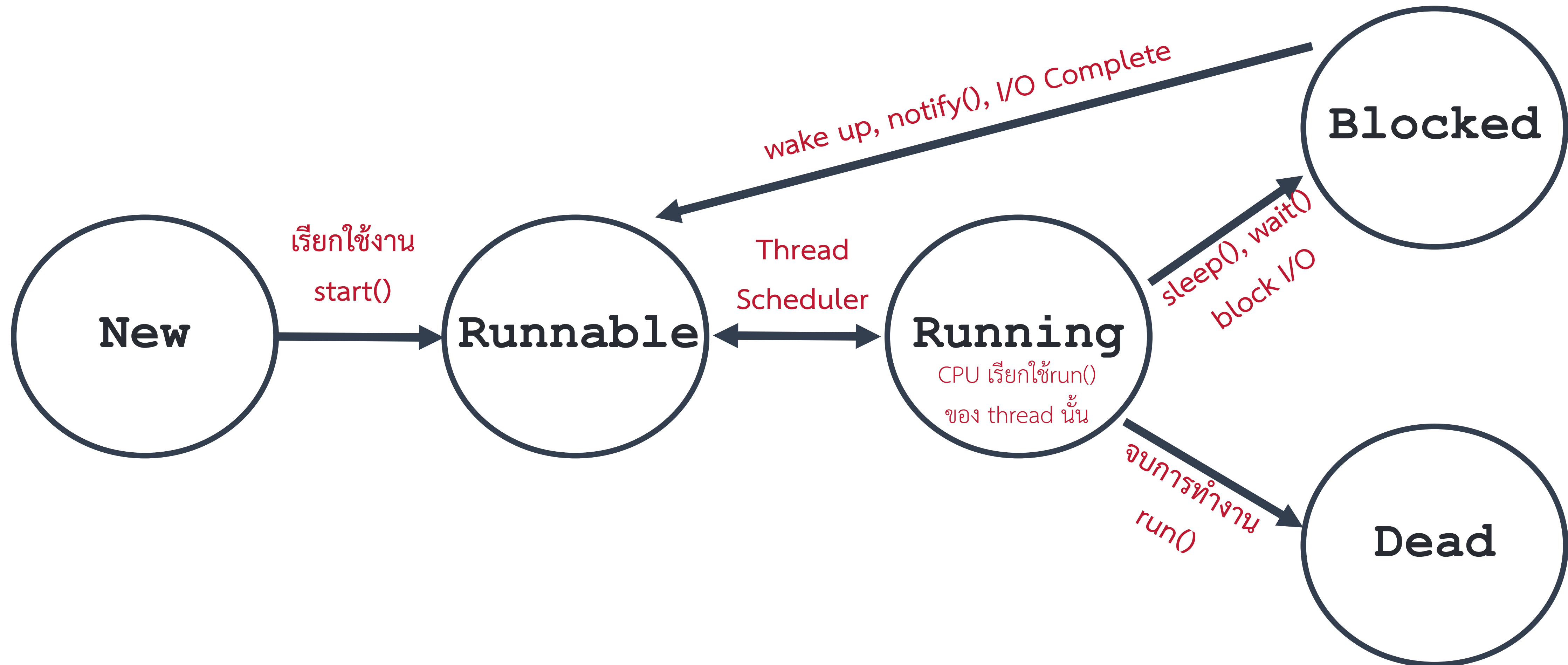
run:
Thana : 0
Thana : 1
Thana : 2
Thana : 3
Somchai : 0
Somchai : 1
Somchai : 2
Somchai : 3
Somchai : 4
Somchai : 5
Somchai : 6
Somchai : 7
Somchai : 8
Thana : 4
Somchai : 9
Somchai : 10
Somchai : 11
Somchai : 12
Somchai : 13
Somchai : 14
Somchai : 15
Somchai : 16
Somchai : 17
Thana : 5
Thana : 6
Thana : 7
  
```


เปรียบเทียบวิธีการสร้างคลาสแบบธรรมดา

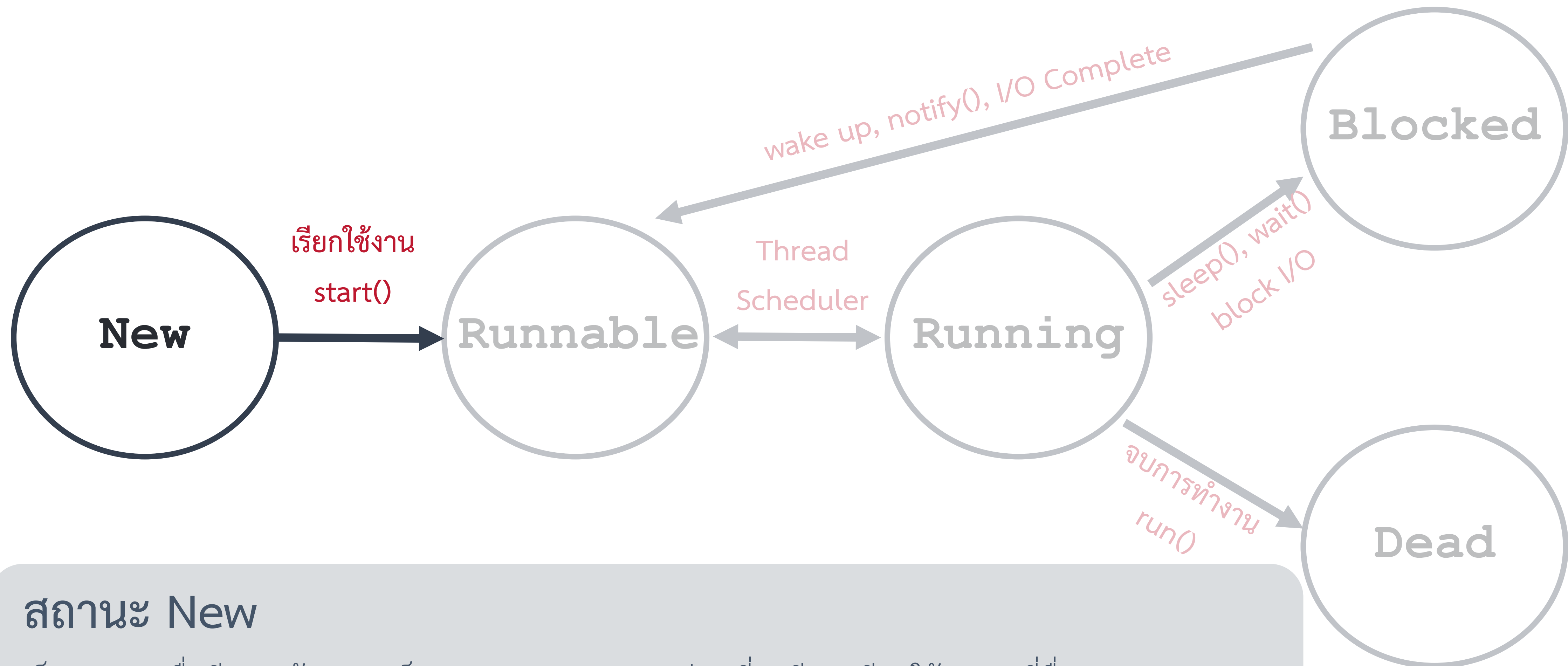
Thread อาจจะเขียน Has C

- การสร้างคลาสโดยการ **สืบทอดคลาสที่ชื่อ Thread** จะเป็นการเขียนโปรแกรมที่สั้นกว่า
- การสร้างคลาสโดย implements **อินเทอร์เฟส Runnable** จะมีหลักการเชิงออปเจ็คที่ดีกว่า
เนื่องจากคลาสดังกล่าว นอกจากนี้ ยังทำให้
การเขียนโปรแกรมมีรูปแบบเดียวกันเสมอ เพราะ **คลาสในจาวาจะสืบทอดได้เพียงคลาสเดียว**

รูปแสดงวงจรการทำงานของเธรด



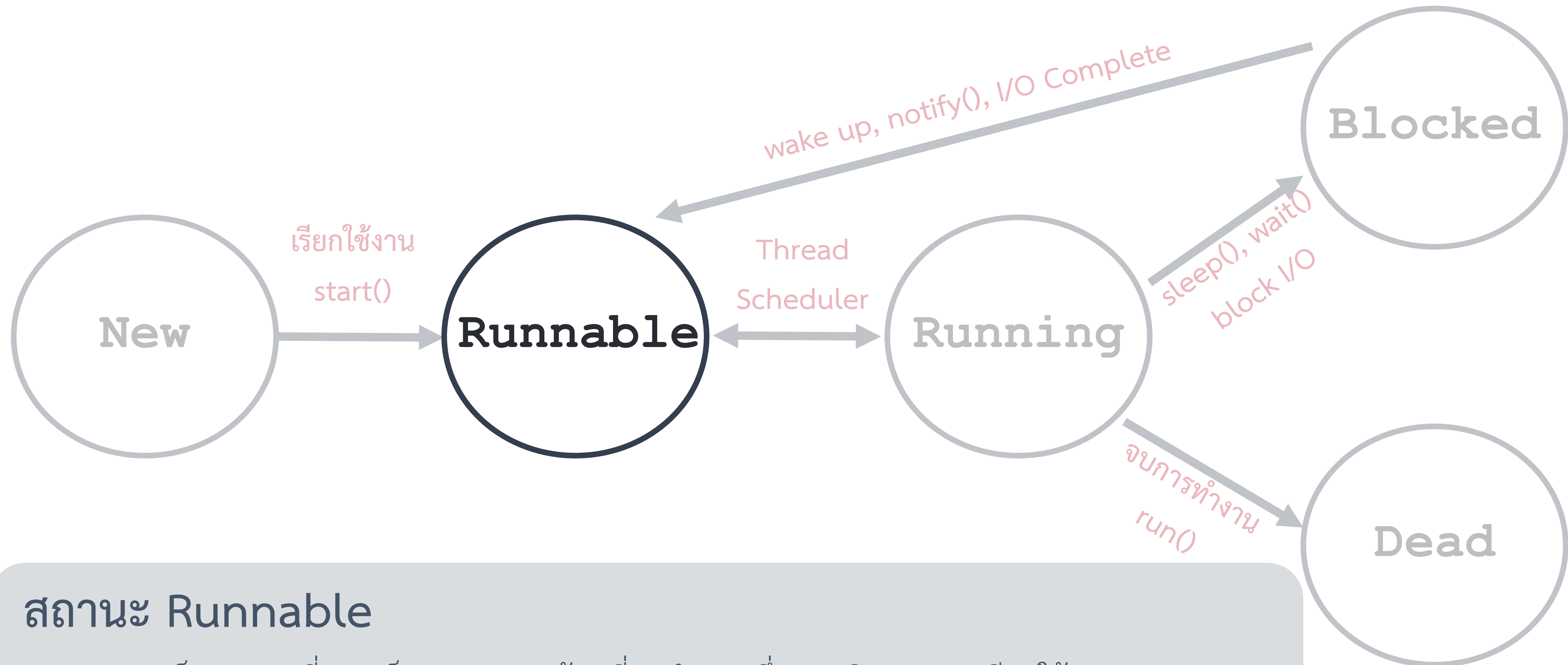
รูปแสดงวงจรการทำงานของเธรด



สถานะ New

เป็นสถานะเมื่อมีการสร้างออบเจ็คของคลาสแบบเธรด ก่อนที่จะมีการเรียกใช้เมธอดที่ชื่อ `start()`

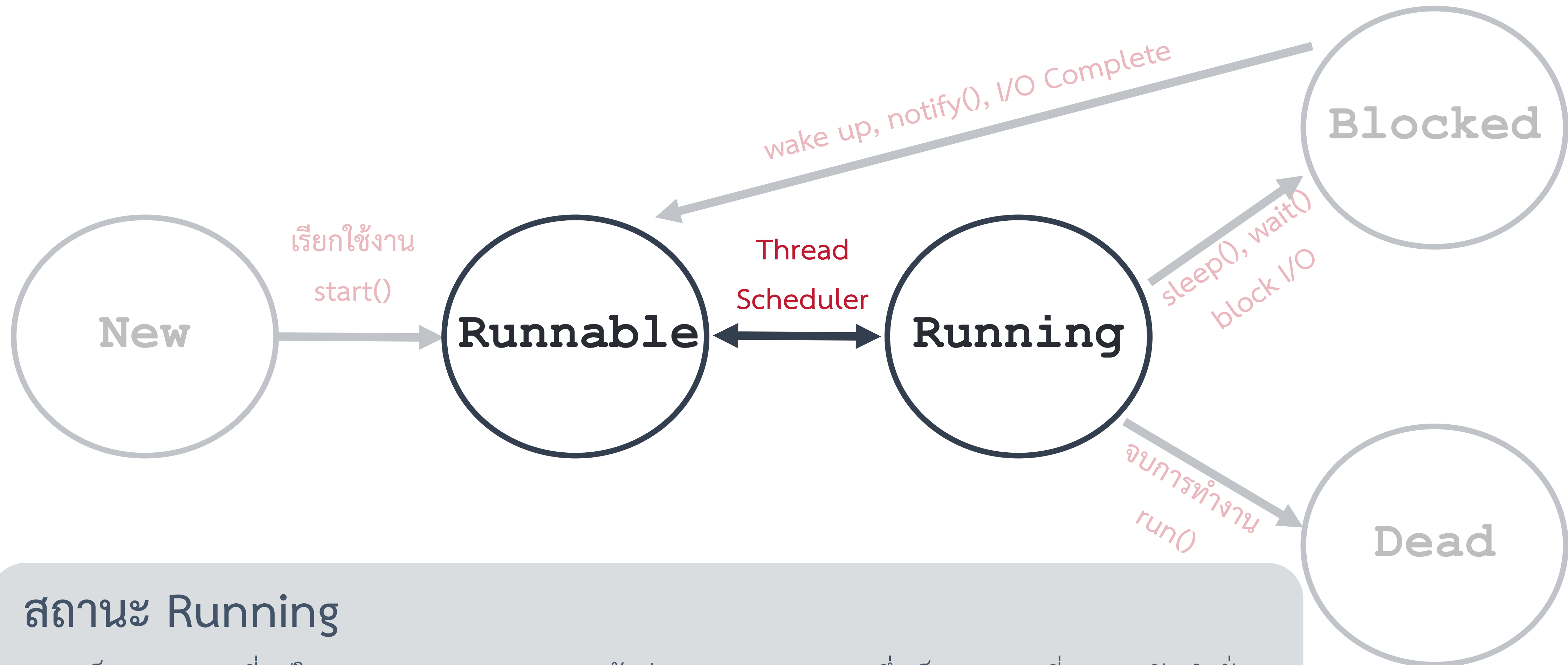
รูปแสดงวงจรการทำงานของเธรด



สถานะ Runnable

Runnable เป็นสถานะที่ออกแบบเธรดพร้อมที่จะทำงาน ซึ่งอาจเกิดจากการเรียกใช้เมธอด `start()` หรือเกิดจากการกลับมาจากสถานะที่เป็นการบล็อกเธรด (Blocked)

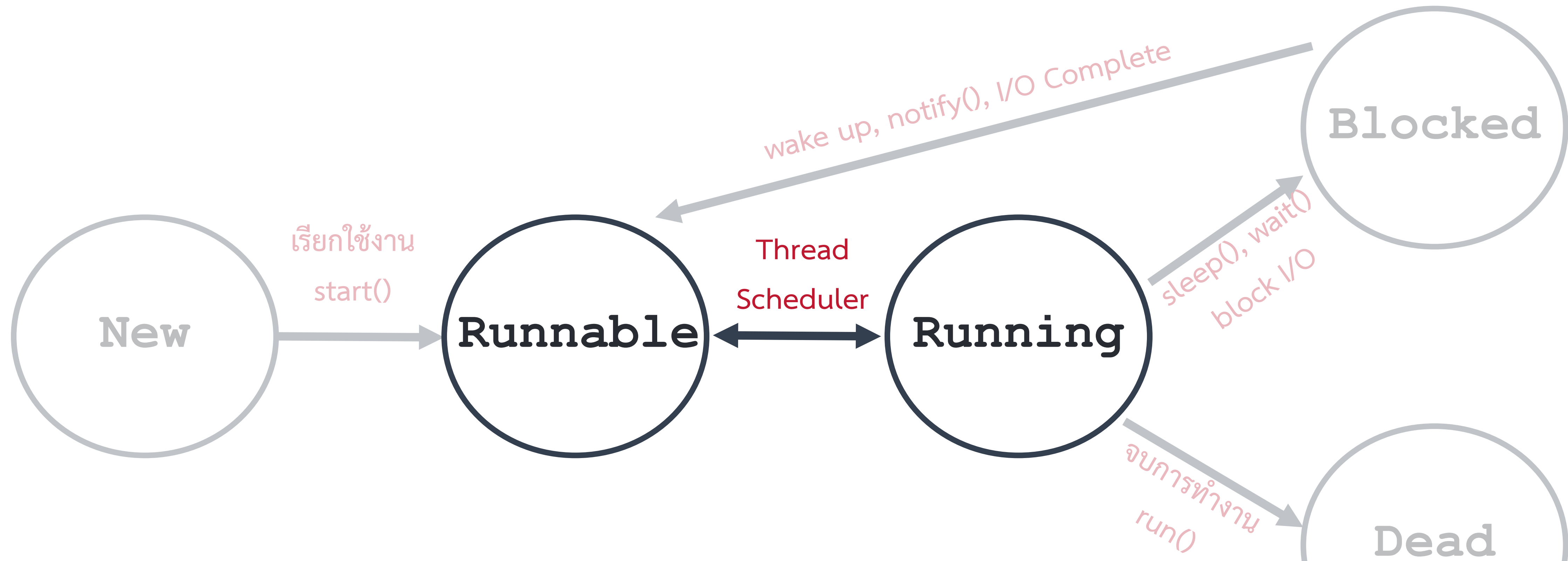
รูปแสดงวงจรการทำงานของเธรด



สถานะ Running

แอปเจ็คแบบเธรดที่อยู่ในสถานะ Runnable จะเข้าสู่สถานะ Running ซึ่งเป็นสถานะที่ CPU รันคำสั่งของแอปเจ็คแบบเธรด เมื่อตัวจัดตารางเวลาจัดเวลาทำงานให้โดยจะทำงานตามชุดคำสั่งในเมธอด run()

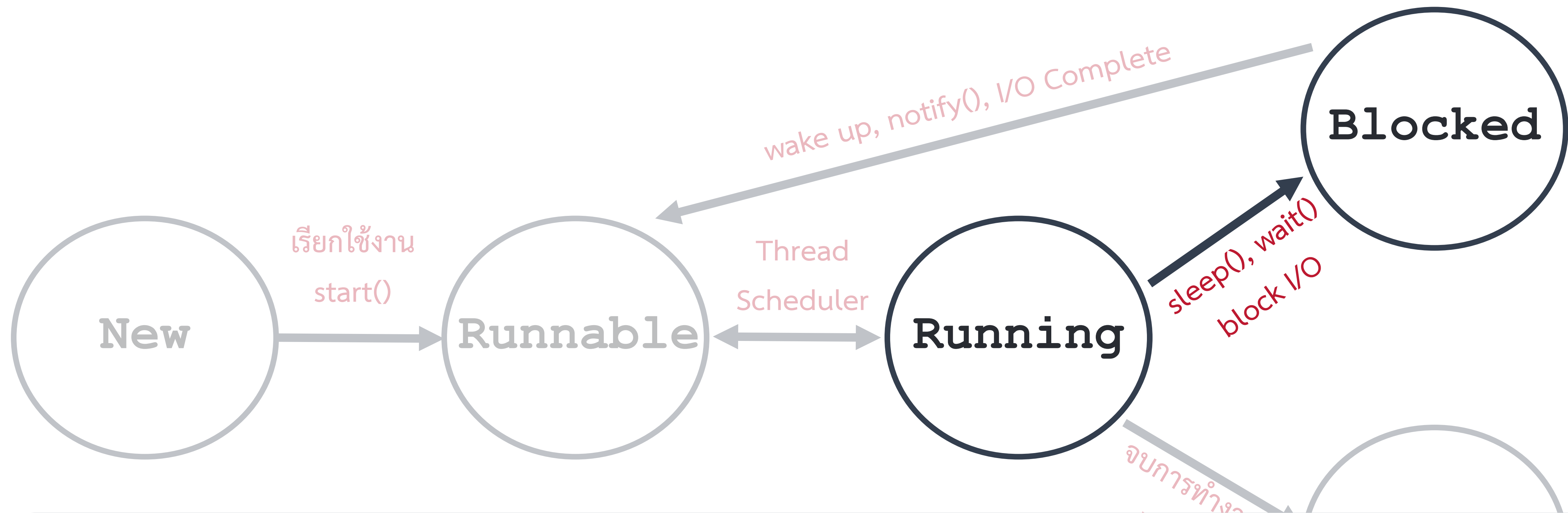
รูปแสดงวงจรการทำงานของเธรด



ความสัมพันธ์ระหว่างสถานะ Runnable และ Running

ออปเจ็คแบบเธรดจะหยุดการทำงานและกลับเข้าสู่สถานะ **Runnable** อีกครั้ง เมื่อตัวจัดตารางเวลาเปลี่ยนให้ออปเจ็คเธรดอื่นที่อยู่ในสถานะ **Runnable** เข้ามาทำงานแทน โดยตัวจัดตารางเวลาจะพยายามจัดการทำงานของออปเจ็คเธรดตามลำดับความสำคัญที่กำหนดไว้

รูปแสดงวงจรการทำงานของเธรด

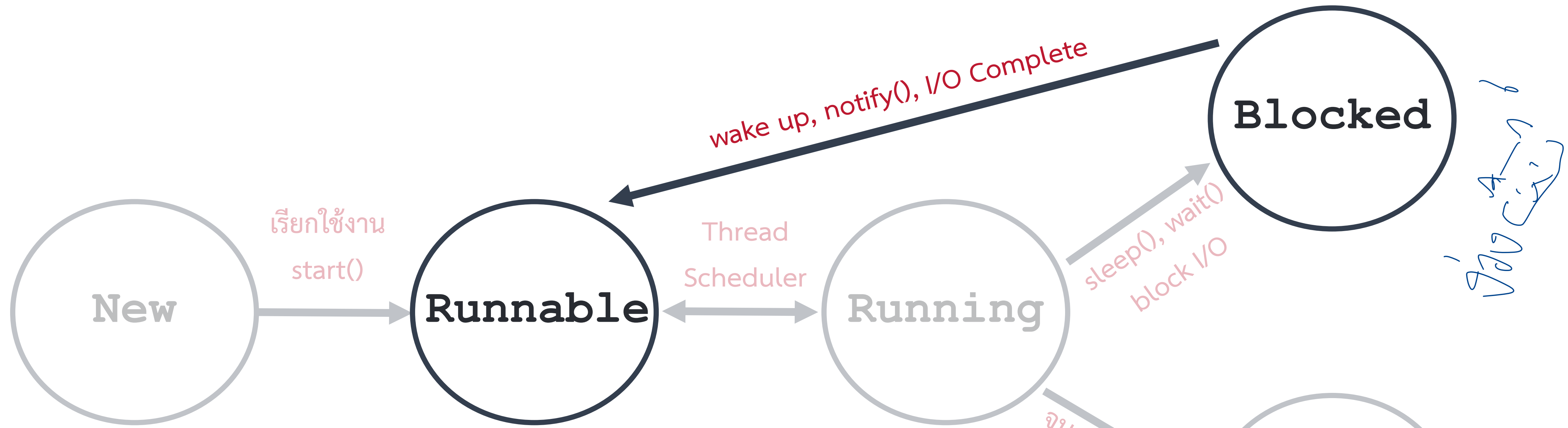


สถานะ Running ไป Blocked

แอปพลิเคชันเธรดที่อยู่ในสถานะ **Running** อาจถูกบล็อกแล้วย้ายไปอยู่ในสถานะ **Blocked** ได้เนื่องจากเหตุการณ์ต่าง ๆ ดังนี้

มีการเรียกใช้เมธอด `sleep()`, แอปพลิเคชันเธรดรอรับการทำงานที่เป็นอินพุตหรือเอาต์พุต, มีการเรียกใช้เมธอด `wait()`, แอปพลิเคชันเธรดถูกเรียกโดยเมธอด `suspend()`

รูปแสดงวงจรการทำงานของเธรด

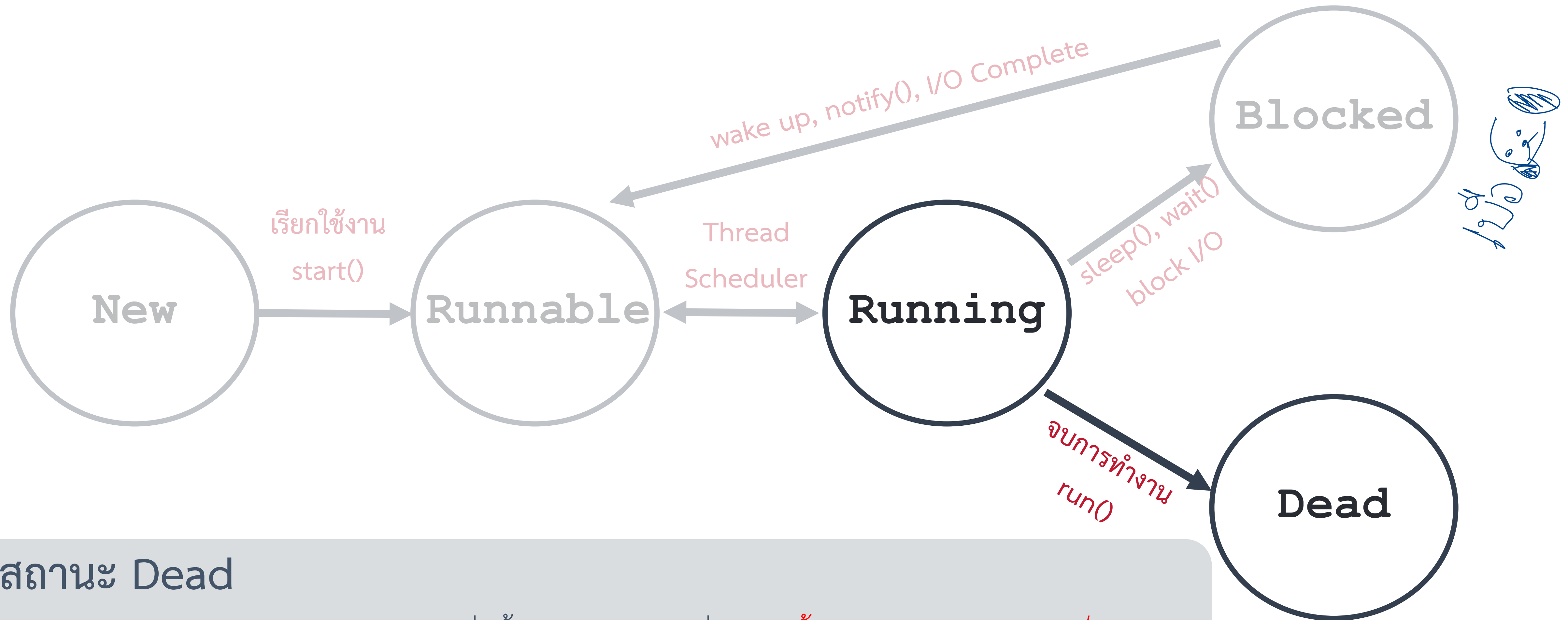


สถานะ Blocked ไป Runnable

ออปเจกแบบเธรดที่อยู่ในสถานะ **Blocked** จะกลับเข้าสู่สถานะ **Runnable** ได้อีกครั้งหนึ่งจากเหตุการณ์ต่าง ๆ ดังนี้

- เมื่อ **หมดระยะเวลาการพัก**ที่กำหนดในเมธอด `sleep()` หรือไม่ได้ถูกขัดจังหวะ (interrupt), เมื่อมี **การส่งข้อมูลอินพุตหรือเอาต์พุต**,
- เมื่อมีออปเจกแบบเธรดตัวอื่นเรียกเมธอด `notify()` หรือ `notifyAll()` ในกรณีที่ออปเจกแบบเธรดถูกบล็อกด้วยเมธอดที่ชื่อ `wait()`,
- เมื่อมีการเรียกใช้เมธอด `resume()` ในกรณีที่ออปเจกแบบเธรดถูกบล็อกด้วยเมธอด `suspend()`

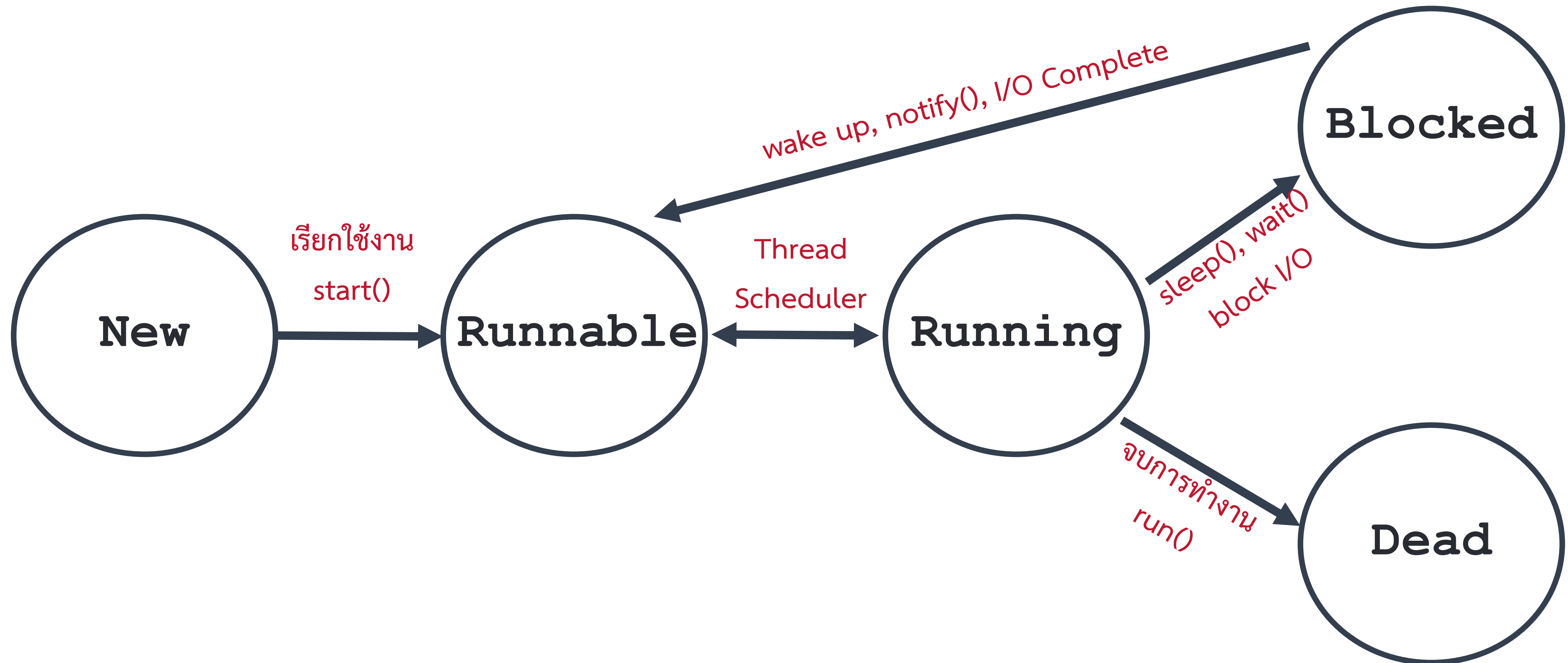
รูปแสดงวงจรการทำงานของเธรด



สถานะ Dead

แอปเจ็คแบบเธรดจะเข้าสู่สถานะ Dead เมื่อสิ้นสุดการทำงานเนื่องจากสิ้นสุดการทำงานในชุดคำสั่งของเมธอด **run()** หรือเมื่อมีการส่งแอปเจ็คประเภท **Exception** ที่ไม่ได้มีการตรวจจับ (catch) ในเมธอด **run()**

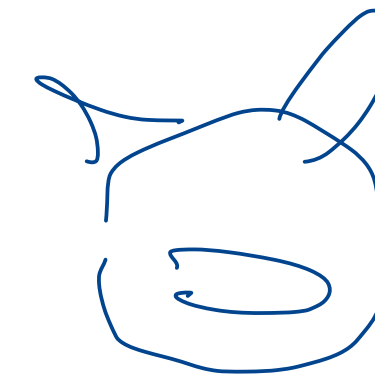
รูปแสดงวงจรการทำงานของเธรด



เมธอดของคลาส Thread

คลาสที่ชื่อ Thread มีเมธอดต่างๆ ที่สำคัญดังนี้

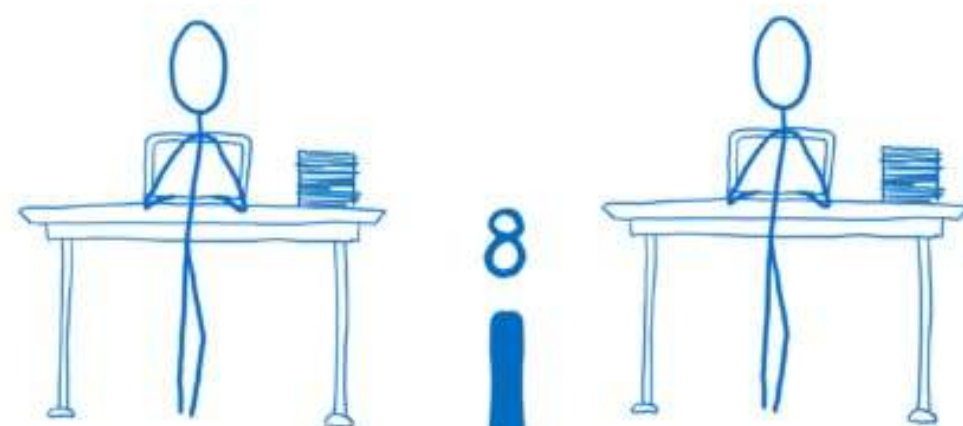
- void run() $\Rightarrow$ *ระบุการทำงานบน Thread ; CPU*
เป็นเมธอดที่จะถูกเรียกใช้เมื่อซีพียูรันโปรแกรมแบบเธรด
- void start() $\rightarrow$ *runable*
เป็นเมธอดที่เรียกใช้งานเพื่อกำหนดให้เธรดเริ่มทำงานโดยเข้าสู่สถานะ *Runnable*
- void suspend()
เป็นเมธอดที่ใช้ในการหยุดการทำงานของเธรด เธรดนี้ไม่มีการแนะนำให้ใช้ในภาษาจาวาตั้งแต่เวอร์ชัน 1.2 เพื่อป้องกันไม่ให้เกิดเหตุการณ์ deadlock ที่จะกล่าวถึงต่อไป
- void resume()
เป็นเมธอดที่ใช้ในการสั่งให้เธรดกลับเข้าสู่สถานะ *Runnable* ภายหลังจากที่ถูกสั่งให้หยุดการทำงาน



ยังไม่เสร็จ

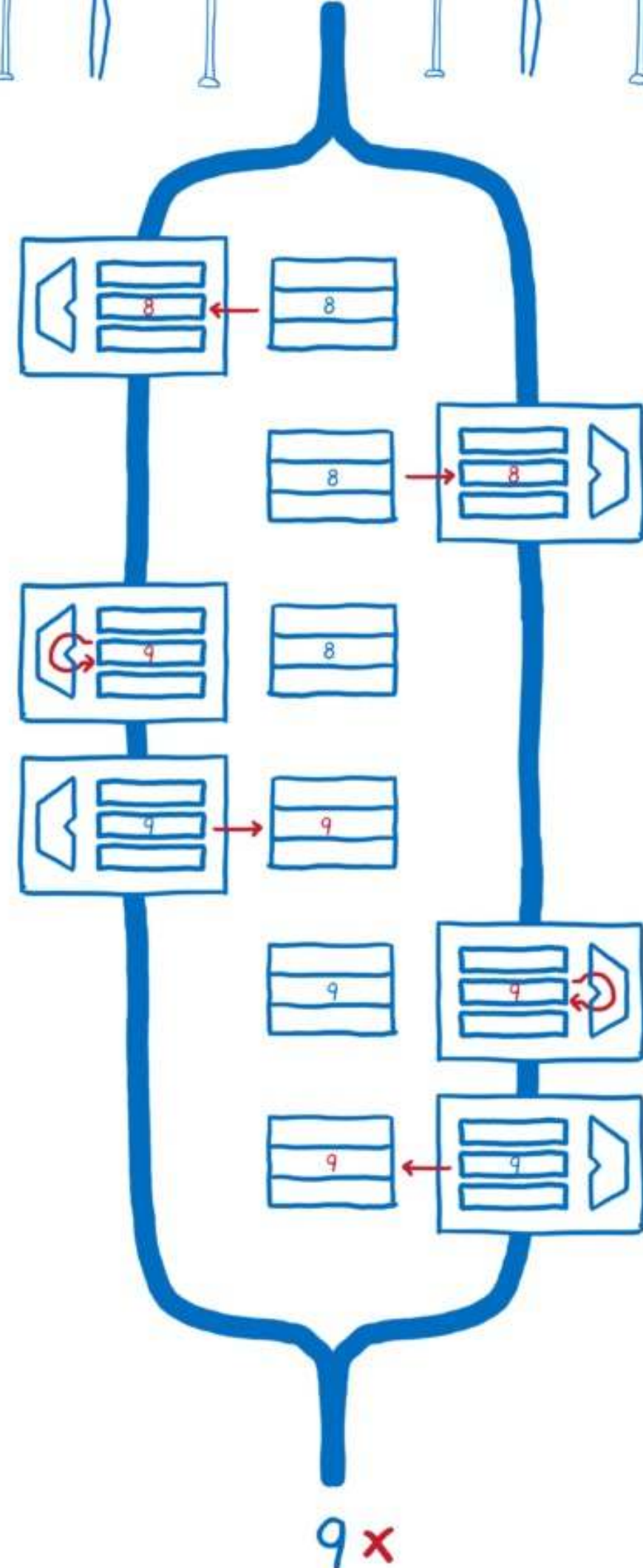
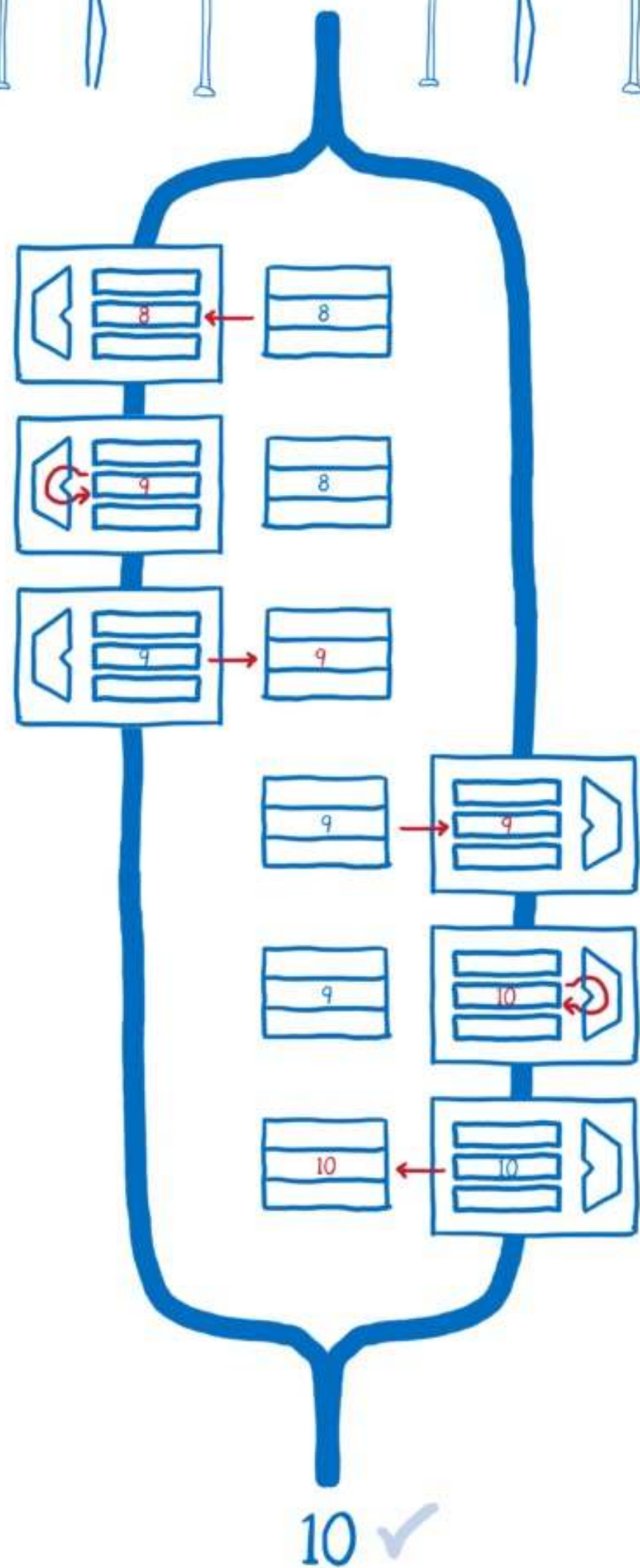
เมธอดของคลาส Thread

- คลาสที่ชื่อ Thread มีเมธอดต่างๆ ที่สำคัญดังนี้
 - static void sleep(long mills) *running → block* *นช่วงเวลานี้*
1000 มิลลิวินาที = 1 วินาที
เป็นเมธอดที่กำหนดให้ *แอปเจ็คแบบเธรดที่อยู่ในสถานะ Running* หยุดการทำงานเป็นเวลา *mills มิลลิวินาที* หรือจนกว่าถูกอินเตอร์รัป
 - boolean isAlive()
เป็นเมธอดที่ใช้ตรวจสอบว่า *แอปเจ็คแบบเธรดยังมีสถานะ Running อยู่หรือไม่*
 - void setPriority(int p)
เป็นเมธอดที่ใช้ในการ *กำหนดลำดับความสำคัญของแอปเจ็คแบบเธรด* ซึ่งจะมีค่าได้ระหว่าง 1 ถึง 10 โดยที่ค่า *10 เป็นค่าสูงสุด*
 - int getPriority()
เป็นเมธอดที่ใช้ในการ *เรียกดูค่าลำดับความสำคัญของแอปเจ็คแบบเธรด*



Synchronization

กรณีที่แอปเจ็คแบบเรดใช้ข้อมูลร่วมกันอาจเกิด
ปัญหาที่เรียกว่า race condition ขึ้นได้ กล่าวคือ
การที่แอปเจ็คแบบเรดต่าง*แย่งกันจัดการข้อมูล*
ทำให้ได้ข้อมูลที่ผิดพลาด ดังตัวอย่าง ต่อไปนี้



ตัวอย่างการเกิด **race conditions** สำหรับการแชร์ทรัพยากรร่วมกัน

ตัวอย่างโปรแกรม

```
class Bank{
    public static final int NTEST = 100;
    private final int[] accounts;
    private long ntransacts = 0;

    public Bank(int n, int initialBalance) {
        accounts = new int[n];
        for (int i = 0; i < accounts.length; i++) {
            accounts[i] = initialBalance;
        }
        ntransacts = 0;
    }

    public void transfer(int from, int to, int amount) {
        if (accounts[from] < amount) {
            return;
        }
        accounts[from] -= amount;
        accounts[to] += amount;
        ntransacts++;
        if (ntransacts % NTEST == 0) {
            test();
        }
    }
}
```

ตัวอย่างโปรแกรม



```
// Class Bank

public void test(){
    int sum = 0;
    for (int i = 0; i < accounts.length; i++){
        sum += accounts[i];
    }
    System.out.println("Transactions:" + ntransacts
                        + " Sum: " + sum);
}

public int size(){
    return accounts.length;
}
}
```


ตัวอย่างโปรแกรม

```
class Transfer extends Thread{
    private Bank bank;
    private int fromAccount;
    private int maxAmount;
    public Transfer(Bank b, int from, int max) {
        bank = b;
        fromAccount = from;
        maxAmount = max;
    }
    public void run() {
        try {
            while (!interrupted()) {
                int toAccount = (int)(bank.size() * Math.random());
                int amount = (int)(maxAmount * Math.random());
                bank.transfer(fromAccount, toAccount, amount);
                Thread.sleep((int) (Math.random() * 10));
            }
        }
        catch (InterruptedException e) {}
    }
}
```

ตัวอย่างโปรแกรม

```
public class BankSystemUnSync{
    public static void main(String[] args){

        final int NACCOUNTS = 10;
        final int INITIAL_BALANCE = 10000;

        Bank b = new Bank(NACCOUNTS, INITIAL_BALANCE);
        int i;
        for (i = 0; i < NACCOUNTS; i++) {
            Transfer t = new Transfer(b, i, INITIAL_BALANCE);
            t.start();
        }
    }
}
```

Output - Lab12 (run) ×

run:

Transactions:100	Sum: 100000
Transactions:202	Sum: 100000
Transactions:301	Sum: 100000
Transactions:400	Sum: 93161
Transactions:500	Sum: 91688
Transactions:601	Sum: 101531
Transactions:700	Sum: 101531
Transactions:802	Sum: 99753
Transactions:900	Sum: 111938
Transactions:1002	Sum: 108832
Transactions:1100	Sum: 108832
Transactions:1201	Sum: 108832
Transactions:1300	Sum: 108832
Transactions:1401	Sum: 111925
Transactions:1500	Sum: 114693
Transactions:1600	Sum: 114010
Transactions:1700	Sum: 114010
Transactions:1801	Sum: 114010
Transactions:1901	Sum: 126569
Transactions:2000	Sum: 126569
Transactions:2100	Sum: 126569

อธิบายตัวอย่างโปรแกรม

เป็นตัวอย่างกรณีของโปรแกรมจำลองระบบธนาคารซึ่งจะมีคลาส Bank ที่มีบัญชีลูกค้าอยู่ 10 บัญชี โดยมียอดเงินรวมของทุกบัญชีทั้งสิ้นจำนวน 100,000 บาท และมีเมธอด transfer() ที่ใช้ในการโอนเงินระหว่างบัญชีหนึ่งไปยังอีกบัญชีหนึ่งโดยใช้คำสั่ง

- `accounts[from] -= amount` (ตัวแปร from คือหมายเลขบัญชีที่ต้องการตัดโอนเงินไป)
- `accounts[to] += amount` (ตัวแปร to คือหมายเลขบัญชีที่ต้องการรับเงินเข้ามา)

คลาส Transfer เป็นคลาสแบบเธรดที่มีเมธอด run() ซึ่งเรียกใช้ออปเจ็คของคลาส Bank เพื่อโอนเงินระหว่างบัญชีใดๆ แบบสุ่มโดยจะทำการโอนเงินซ้ำไปไม่มีที่สิ้นสุด และทุกๆ การโอนเงินจำนวน 100 ครั้ง จะพิมพ์ยอดเงินรวมของบัญชีทุกบัญชีออกมา ซึ่งควรจะมีค่าเป็น 100,000 บาท

อธิบายผลลัพธ์ที่ได้จากการรันโปรแกรม

ตัวอย่างผลลัพธ์ที่ได้จากการรันโปรแกรมนี จะเห็นได้ว่าเมื่อรันโปรแกรมไปได้ระยะหนึ่ง จะมี**ผลรวมยอดเงินที่ผิดพลาด** ทั้งนี้เนื่องจากเกิดปัญหาที่เรียกว่า race condition ซึ่งออกแบบแบบเธรดแต่ละตัวอาจหยุดทำงานระหว่างขั้นตอนใดๆ ของคำสั่งในเมธอด run() เพื่อให้ออกแบบเธรดตัวอื่นได้ทำงาน ซึ่งคำสั่งแต่ละคำสั่งอาจมีขั้นตอนการทำงานหลายขั้น เช่นคำสั่ง

`account[to] += amount`

จะไม่ใช้เป็นเพียงคำสั่งเดียว แต่จะมีขั้นตอนต่างๆดังนี้

- โหลด `account[to]` เข้าสู่รีจิสเตอร์ในซีพียู
- บวกค่าในรีจิสเตอร์ด้วยค่าของ `amount`
- มีผลลัพธ์จากรีจิสเตอร์กลับมายัง `account[to]`

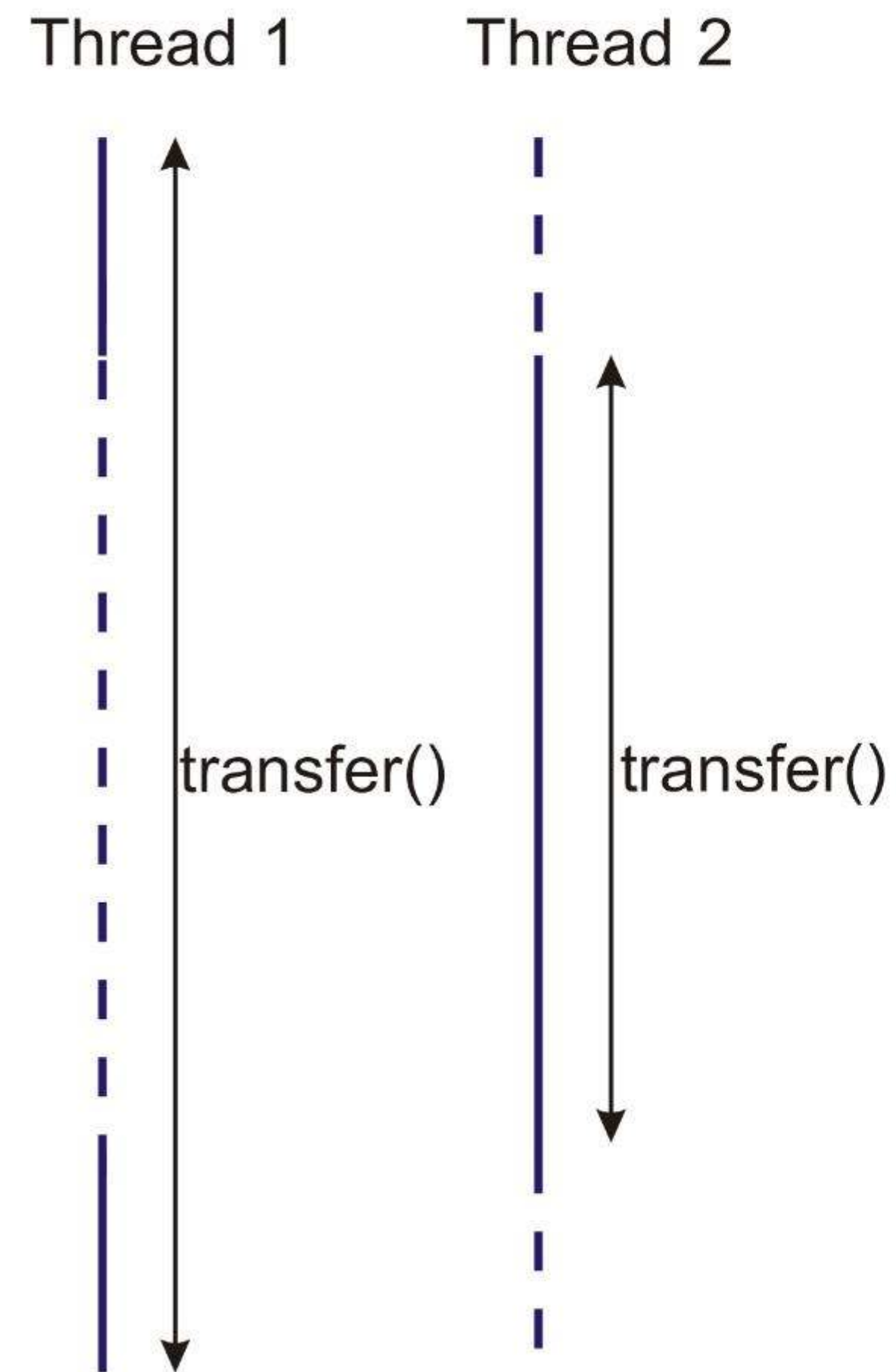
คีย์เวิร์ด synchronized

การที่ออปเจ็คแบบเธรดหยุดการทำงานในขั้นตอนใดขั้นตอนหนึ่ง เพื่อเปิดโอกาสให้ออปเจ็คแบบเธรดตัวอื่นๆ เข้ามาทำงาน ก็อาจจะทำให้ได้ผลลัพธ์ผิดพลาดได้ เราสามารถใช้คีย์เวิร์ด **synchronized** เพื่อล็อก (lock) ชุดคำสั่งที่ออปเจ็คแบบเธรดต้องกระทำพร้อมกันไว้ได้ ซึ่งส่งผลให้ออปเจ็คแบบเธรดจะไม่หยุดการทำงานระหว่างบล็อกคำสั่งที่ถูกล็อกไว้ และออปเจ็คแบบเธรดตัวอื่น ๆ จะต้องรอให้ซีพียูทำชุดคำสั่งที่ถูกล็อกไว้ให้เสร็จก่อนแล้วจึงเข้ามาทำงานต่อได้ อย่างไรก็ตาม ออปเจ็คแบบเธรดจะปลดล็อกเมื่อ

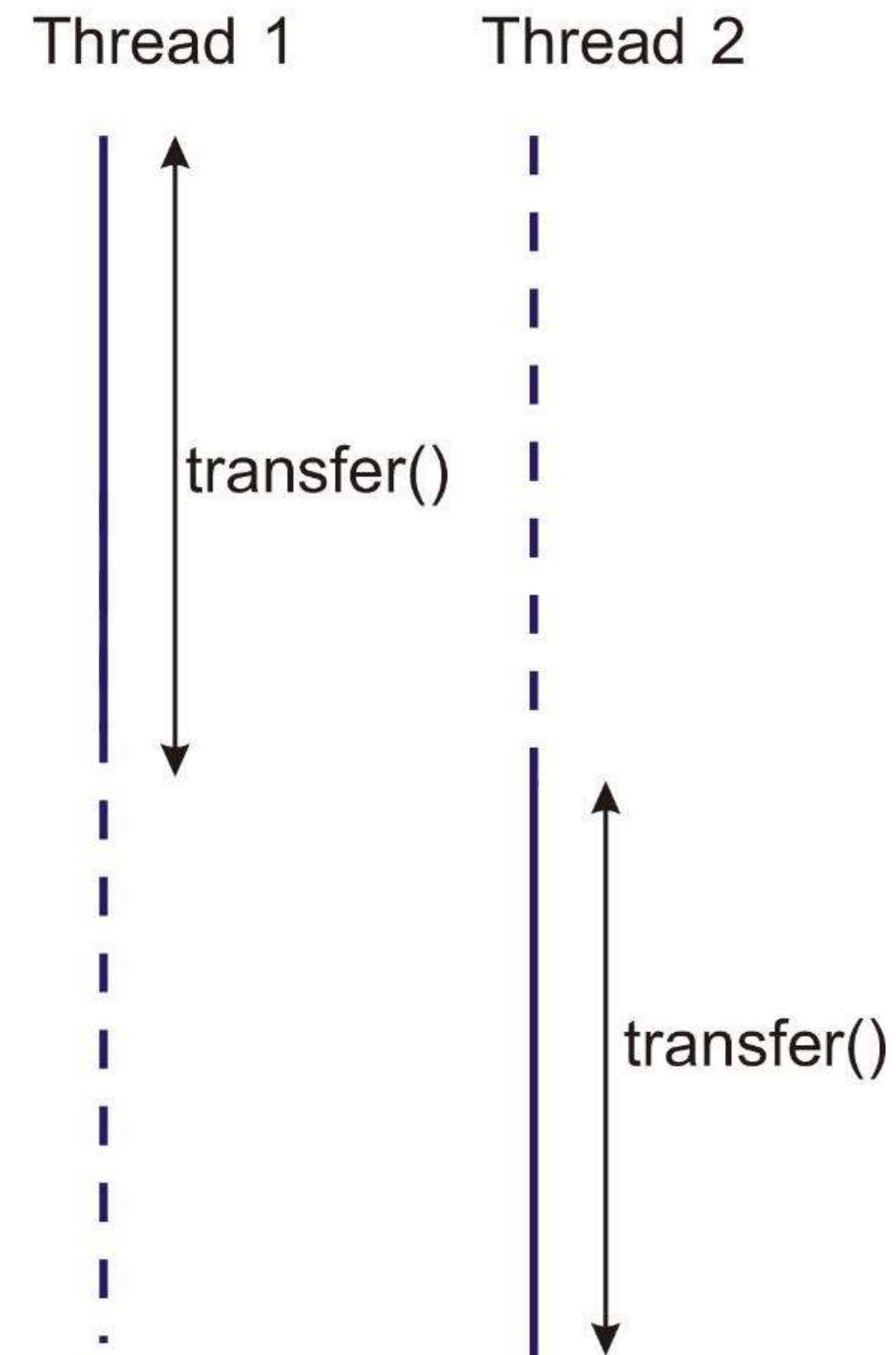
- สิ้นสุดคำสั่งที่ใช้บล็อกคำสั่งของ **synchronized** หรือ
- เมื่อได้รับข้อผิดพลาดจากออปเจ็คแบบ **Exception**

รูปแสดงการเปรียบเทียบ

Unsynchronized



Synchronized



การใช้คีย์เวิร์ด `synchronized`

การใช้คีย์เวิร์ด `synchronized` เพื่อล็อกคำสั่งทำได้ 2 รูปแบบคือ

- กำหนดคีย์เวิร์ด `synchronized` ที่เมธอด

```
public synchronized void transfer(int from, int to, int amount) {  
    ...  
}
```

- กำหนดบล็อกคีย์เวิร์ด `synchronized`

```
public void increase() {  
    synchronized (this) {    // this คือ monitor object  
        ...  
    }  
}
```

ตัวอย่างของการกำหนดเมธอดที่ชื่อ `transfer()` ให้เป็นแบบ `synchronized` สามารถทำได้ดังนี้

ตัวอย่างการใช้คีย์เวิร์ด `synchronized`

```
class Bank {  
    public static final int NTEST = 10000;  
    private final int[] accounts;  
    private long ntransacts = 0;  
  
    public Bank(int n, int initialBalance) {  
        accounts = new int[n];  
        for (int i = 0; i < accounts.length; i++) {  
            accounts[i] = initialBalance;  
        }  
        ntransacts = 0;  
    }  
    public synchronized void transfer(int from, int to, int amount) {  
        try {  
            accounts[from] -= amount;  
            accounts[to] += amount;  
            ntransacts++;  
            if (ntransacts % NTEST == 0) {  
                test();  
            }  
        } catch (InterruptedException e) {}  
    }  
}
```

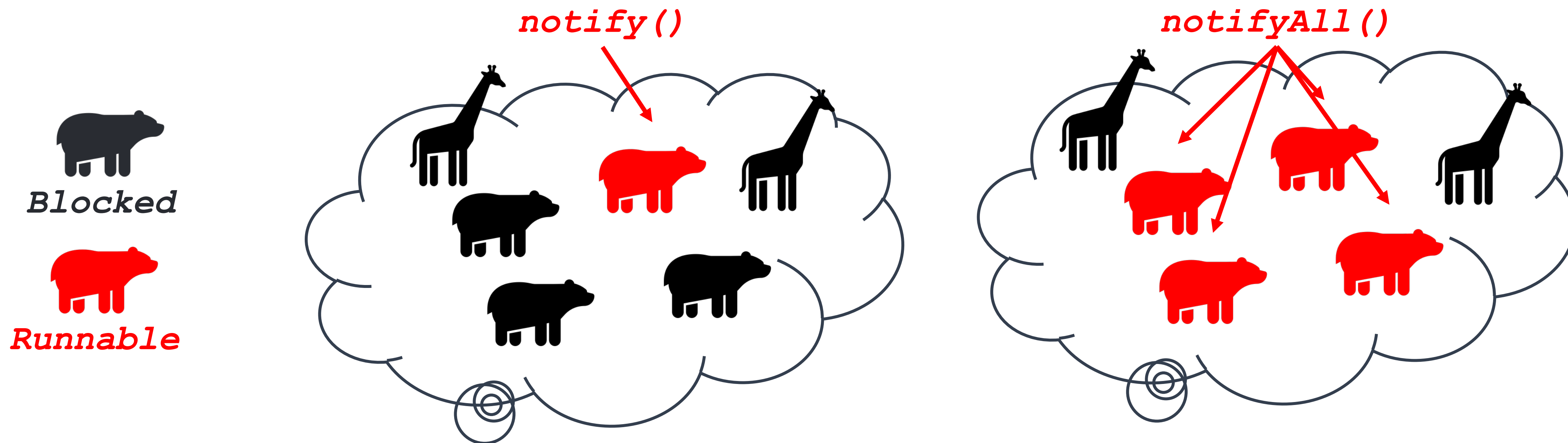
Output - Lab12 (run) X

```
run:  
Transactions:100 Sum: 100000  
Transactions:200 Sum: 100000  
Transactions:300 Sum: 100000  
Transactions:400 Sum: 100000  
Transactions:500 Sum: 100000  
Transactions:600 Sum: 100000  
Transactions:700 Sum: 100000  
Transactions:800 Sum: 100000  
Transactions:900 Sum: 100000  
Transactions:1000 Sum: 100000
```

```
public synchronized void test() {  
    int sum = 0;  
    for (int i = 0; i < accounts.length; i++){  
        sum += accounts[i];  
    }  
    System.out.println("Transactions:" + ntransacts  
                        + " Sum: " + sum);  
}
```


เมธอด wait() และ notify()

- เมธอด wait() และ notify() หรือ notifyAll() เป็นเมธอดที่ใช้กับเมธอดหรือบล็อกคำสั่งที่เป็น synchronized
- เมธอด wait() จะมีผลให้ออปเจ็คแบบเรดที่อยู่ในสถานะ *Running* กลับเข้าสู่สถานะ *Blocked* เพื่อรอให้ออปเจ็คแบบเรดตัวอื่น ๆ *แจ้งให้กลับเข้าสู่สถานะ Runnable* โดยใช้เมธอด *notify()* หรือ *notifyAll()*



ตัวอย่างการใช้คีย์เวิร์ด synchronized

```

class Bank {
    public static final int NTEST = 10000;
    private final int[] accounts;
    private long ntransacts = 0;

    public Bank(int n, int initialBalance) {
        accounts = new int[n];
        for (int i = 0; i < accounts.length; i++) {
            accounts[i] = initialBalance;
        }
        ntransacts = 0;
    }

    public synchronized void transfer(int from, int to, int amount) {
        try {
            while (accounts[from] < amount) {
                wait();
            }
            accounts[from] -= amount;
            accounts[to] += amount;
            ntransacts++;
            notifyAll();
            if (ntransacts % NTEST == 0) {
                test();
            }
        } catch (InterruptedException e) {}
    }
}
  
```

Output - Lab12 (run) X

```

run:
Transactions:100 Sum: 100000
Transactions:200 Sum: 100000
Transactions:300 Sum: 100000
Transactions:400 Sum: 100000
Transactions:500 Sum: 100000
Transactions:600 Sum: 100000
Transactions:700 Sum: 100000
Transactions:800 Sum: 100000
Transactions:900 Sum: 100000
Transactions:1000 Sum: 100000
  
```

```

public synchronized void test() {
    int sum = 0;
    for (int i = 0; i < accounts.length; i++) {
        sum += accounts[i];
    }
    System.out.println("Transactions:" + ntransacts
        + " Sum: " + sum);
}
  
```



```

public class LblCount extends JLabel implements Runnable{
    private int time;
    private boolean paused;
    private synchronized void checkPaused(){
        while (paused){
            try{
                this.wait();
            }
            catch (InterruptedException ex){ ex.printStackTrace(); }
        }
    }
    public void pauseThread(){ paused = true; }
    public synchronized void resumeThread(){
        paused = false;
        this.notify();
    }
    public void run() {
        try {
            while (true) {
                checkPaused();
                setText(time+"");
                time += 1;
                Thread.sleep(1000);
                System.out.println(Thread.currentThread().getName());
            }
        } catch (InterruptedException ex){ ex.printStackTrace(); }
    }
}

```

Handwritten notes:

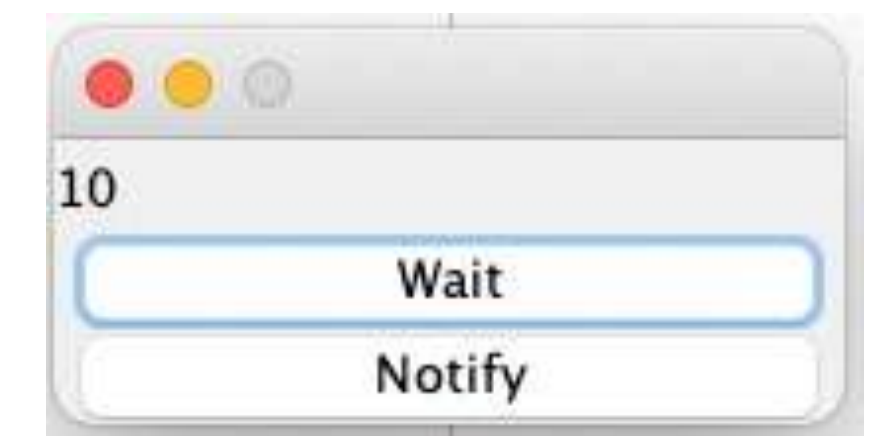
- ⇒ เมื่อ wait* (next to `paused = true;`)
- ⇒ เมื่อ notify* (next to `resumeThread()`)

ข้อควรระวัง

เมธอด `wait()` จะไม่หยุด Object ที่นักศึกษาเรียกใช้ แต่มันจะทำให้เธรดปัจจุบันเข้าสู่สถานะ Waiting แทน เพื่อหลีกเลี่ยงการปัญหาข้างต้น นักศึกษาควร

- ย้ายการเรียก `wait()` ไปยังภายในเธรดตัวนั้น
- ต้องให้ตัวมันเรียกตัวมันเอง

ลิงค์ดูตัวอย่าง <https://youtu.be/2mDKWaobxGU>



```

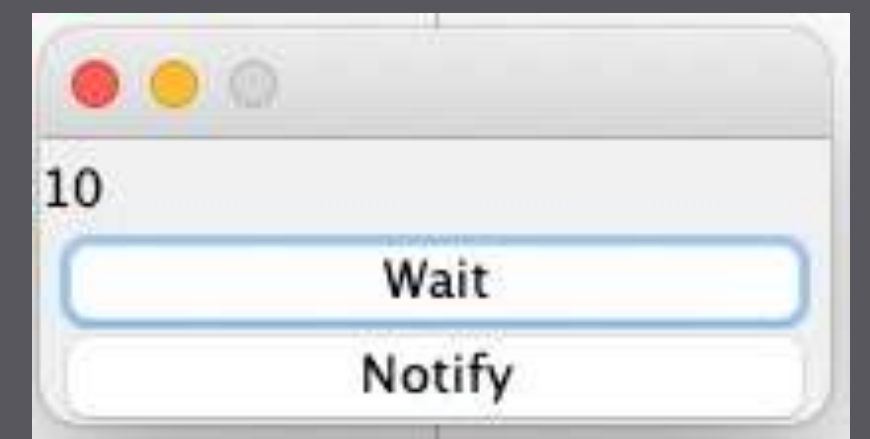
public class MyCount implements ActionListener {
    private JFrame mainFrame;      private JPanel mainPanel;
    private JLabel lblCount;      private JButton btn1, btn2;      private Thread t1;
    public MyCount() {
        mainFrame = new JFrame();    mainPanel = new JPanel();    lbl = new JLabel();
        btn1 = new JButton("Wait");  btn2 = new JButton("Notify");
        t1 = new Thread(lbl);

        mainFrame.add(mainPanel);
        mainPanel.setLayout(new GridLayout(3,1));
        mainPanel.add(lbl);          mainPanel.add(btn1);          mainPanel.add(btn2);

        btn1.addActionListener(this);    btn2.addActionListener(this);
        mainFrame.setSize(200, 100);
        mainFrame.setDefaultCloseOperation(WindowConstants.EXIT_ON_CLOSE);
        mainFrame.setResizable(false);  mainFrame.setVisible(true);
        t1.start();
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        if (e.getSource().equals(btn2)) {    lbl.resumeThread();    }
        else if (e.getSource().equals(btn1)) {    lbl.pauseThread();    }
    }
}

```



ลิงค์ดูตัวอย่าง <https://youtu.be/2mDKWaobxGU>

```

public class LblCount2 extends JLabel implements Runnable{
    private int time;
    private boolean paused;
    private synchronized void pauseThread(){
        paused = true;
        while (paused){
            try{
                this.wait();
            }
            catch (InterruptedException ex){ ex.printStackTrace(); }
        }
    }
    public synchronized void resumeThread(){
        paused = false;
        this.notify();
    }
    public void run() {
        try {
            while (true) {
                setText(time+"");
                time += 1;
                Thread.sleep(1000);
                System.out.println(Thread.currentThread().getName());
            }
        }catch (InterruptedException ex){ ex.printStackTrace(); }
    }
}

```

ข้อควรระวัง

เมธอด wait() จะไม่หยุด Object ที่นักศึกษาเรียกใช้ แต่มันจะทำให้เธรดปัจจุบันเข้าสู่สถานะ Waiting แทน เพื่อหลีกเลี่ยงการปัญหาข้างต้น นักศึกษาควร

- ย้ายการเรียก wait() ไปยังภายในเธรดตัวนั้น
- ต้องให้ตัวมันเรียกตัวมันเอง

อ้างอิง <https://coderanch.com/t/473458/java/wait-freezing-gui>

Deadlock



- ในกรณีที่ออปเจ็คแบบเธรดทุกออปเจ็คถูกเรียกใช้คำสั่ง wait() ทั้งหมดโดยไม่มีออปเจ็คใดสามารถเรียกคำสั่ง notify() หรือ notifyAll() ได้ จะเป็นกรณีที่ **ออปเจ็คแต่ละตัวรอการปลดล็อกจากกันและกัน** จึงทำให้เกิดสถานการณ์ที่เรียกว่า deadlock ซึ่งจะทำให้ไม่มีการทำงานใดๆ เกิดขึ้น
- ตัวอย่างเช่น ถ้าคลาส Bank ของโปรแกรมที่ผ่านมามีบัญชีเพียงสองบัญชีโดยบัญชีที่หนึ่งมียอดเงิน 2,000 บาท และบัญชีที่สองมียอดเงิน 3,000 บาท และถ้าออปเจ็คแบบเธรดตัวที่หนึ่งมีคำสั่ง transfer() เพื่อโอนเงินจากบัญชีที่หนึ่งไปยังบัญชีที่สองเป็นเงิน 3,000 บาท และออปเจ็คแบบเธรดตัวที่สองมีคำสั่ง transfer() เพื่อโอนเงินจากบัญชีที่สองไปยังบัญชีที่หนึ่งเป็นเงิน 4,000 บาท กรณีนี้จะเกิดเหตุการณ์ deadlock ขึ้น
- การป้องกันการเกิด deadlock สามารถทำได้โดยการจัดลำดับการล็อกออปเจ็คแบบเธรดให้ถูกต้อง

รูปแสดงเหตุการณ์ Deadlock

